Практическое занятие №9. Реализация регистрации и входа пользователей. Хэширование паролей с bcrypt

Дисциплина: Технологии индустриального программирования
Язык программирования: Go
База данных: PostgreSQL
ORM: GORM
HTTP-маршрутизатор: chi
Алгоритм хэширования: bcrypt

1. Цель работы

Цель практического занятия — реализовать безопасное хранение паролей пользователей, регистрацию и вход в систему, проверку входных данных и обработку типовых ошибок. Для хранения учётных записей используется PostgreSQL, для взаимодействия с базой данных — GORM, для хэширования паролей — bcrypt.

В результате работы должен получиться HTTP-сервис `pz9-auth` со следующими маршрутами:

- `POST /auth/register` — регистрация пользователя по email и паролю;
- `POST /auth/login` — проверка email и пароля.

При успешной регистрации сервер сохраняет email и bcrypt-хэш пароля. При успешном входе возвращает ответ со статусом `ok`. JWT в рамках этой работы не создаётся: его добавление предусмотрено в следующем практическом занятии.

2. Краткие теоретические сведения

Пароли нельзя хранить в базе данных в открытом виде. Если база данных окажется скомпрометирована, злоумышленник сможет получить исходные пароли пользователей. Риск становится выше, если пользователь применяет один пароль на нескольких сайтах.

Шифрование и хэширование отличаются принципом работы. Зашифрованные данные можно восстановить при наличии ключа. Хэширование является односторонним преобразованием: приложение сохраняет результат обработки пароля и использует его для проверки при входе.

Для хранения паролей применяется bcrypt. Алгоритм автоматически генерирует случайную соль и сохраняет её вместе с результатом хэширования. Поэтому два

одинаковых пароля, обработанные отдельно, обычно дают разные строки хэша. Для проверки используется `bcrypt.CompareHashAndPassword`: функция самостоятельно учитывает соль и параметры, записанные в хэше.

Параметр `cost` определяет вычислительную сложность bcrypt. Чем выше значение, тем больше времени занимает создание и проверка хэша. В учебной работе используется `cost = 12`. Быстрые хэш-функции общего назначения, например MD5 или SHA-256, для непосредственного хранения паролей не подходят: они позволяют выполнять слишком много попыток подбора за короткое время.

3. Подготовка рабочего окружения

Для выполнения работы потребуются:

- Go версии 1.23 или новее;
- PostgreSQL;
- Docker Desktop с Docker Compose либо локально установленный PostgreSQL;
- редактор кода, например Visual Studio Code или GoLand;
- PowerShell в Windows. В Linux/macOS команды выполняются в терминале.

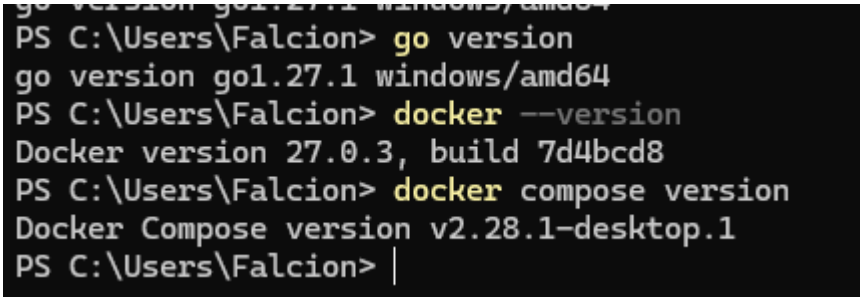
Проверим установленную версию Go:

```
go version
```

Для проверки Docker:

```
docker --version
docker compose version
```

В Linux/macOS команды такие же.



4. Создание проекта и установка зависимостей

Сначала создадим рабочую папку и перейдём в неё. В Windows PowerShell:

```
mkdir pz9-auth
cd pz9-auth
```

Инициализируем Go-модуль:

```
go mod init example.com/pz9-auth
```

Затем установим необходимые библиотеки:

```
go get github.com/go-chi/chi/v5
go get gorm.io/gorm
go get gorm.io/driver/postgres
go get golang.org/x/crypto/bcrypt
```

```
PS D:\.WORKSPACE\Education\T1.9> cd .\pz9-auth\
PS D:\.WORKSPACE\Education\T1.9\pz9-auth> go mod init example.com/pz9-auth
go: creating new go.mod: module example.com/pz9-auth
PS D:\.WORKSPACE\Education\T1.9\pz9-auth> go get github.com/go-chi/chi/v5
go: added github.com/go-chi/chi/v5 v5.5.0
PS D:\.WORKSPACE\Education\T1.9\pz9-auth> go get gorm.io/gorm
go: added github.com/jinzhu/inflection v1.0.0
go: added github.com/jinzhu/now v1.1.5
go: added golang.org/x/text v0.20.0
go: added gorm.io/gorm v1.31.2
PS D:\.WORKSPACE\Education\T1.9\pz9-auth> go get gorm.io/driver/postgres
go: added github.com/jackc/pgpassfile v1.0.0
go: added github.com/jackc/pgservicefile v0.0.0-20240606120523-5a60cdf6a761
go: added github.com/jackc/pgx/v5 v5.10.0
go: added github.com/jackc/puddle/v2 v2.2.2
go: upgraded golang.org/x/sync v0.9.0 => v0.17.0
go: upgraded golang.org/x/text v0.20.0 => v0.29.0
go: added gorm.io/driver/postgres v1.6.3
PS D:\.WORKSPACE\Education\T1.9\pz9-auth> go get golang.org/x/crypto/bcrypt
```

5. Создание структуры каталогов

В корне проекта создадим каталоги для точки входа приложения, модели пользователя, HTTP-обработчиков, конфигурации и репозитория:

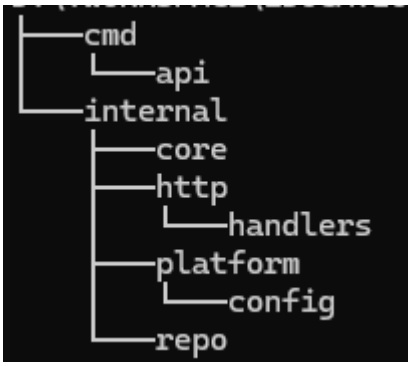
```
mkdir -Force cmd/api, internal/core, internal/http/handlers,
internal/platform/config, internal/repo | Out-Null
```

Структура проекта будет следующей:

```
pz9-auth/
├── cmd/
```

```
│   └── api/
│       └── main.go
├── internal/
│   ├── core/
│   │   └── user.go
│   ├── http/
│   │   └── handlers/
│   │       └── auth.go
│   ├── platform/
│   │   └── config/
│   │       └── config.go
│   └── repo/
│       ├── postgres.go
│       └── user_repo.go
└── go.mod
```

Каталог `cmd/api` содержит точку входа приложения. В `internal/core` располагается модель пользователя. В `internal/http/handlers` находятся HTTP-обработчики. Каталог `internal/platform/config` отвечает за чтение настроек, а `internal/repo` — за подключение к PostgreSQL и операции с пользователями.



6. Настройка конфигурации приложения

Создадим файл `internal/platform/config/config.go`. В нём будут считываться три переменные окружения:

- `DB_DSN` — строка подключения к PostgreSQL;
- `BCRYPT_COST` — сложность bcrypt;
- `APP_ADDR` — адрес и порт HTTP-сервера.

Содержимое файла:

```
package config

import (
```

```
        "fmt"
        "os"
        "strconv"

        "golang.org/x/crypto/bcrypt"
)

const defaultBcryptCost = 12
const maxRecommendedBcryptCost = 14

type Config struct {
        DBDSN      string
        BcryptCost int
        Addr       string
}

func Load() (Config, error) {
        dsn := os.Getenv("DB_DSN")
        if dsn == "" {
                return Config{}, fmt.Errorf("DB_DSN is required")
        }

        cost := defaultBcryptCost
        if value := os.Getenv("BCRYPT_COST"); value != "" {
                parsed, err := strconv.Atoi(value)
                if err != nil {
                        return Config{}, fmt.Errorf("BCRYPT_COST must be an
integer: %w", err)
                }
                cost = parsed
        }
        if cost < bcrypt.MinCost || cost > maxRecommendedBcryptCost {
                return Config{}, fmt.Errorf(
                        "BCRYPT_COST must be between %d and %d for this
project",
                        bcrypt.MinCost,
                        maxRecommendedBcryptCost,
                )
        }

        addr := os.Getenv("APP_ADDR")
        if addr == "" {
                addr = ":8080"
        }

        return Config{
```

```
                DBDSN:      dsn,
                BcryptCost: cost,
                Addr:       addr,
        }, nil
}
```

Если `BCRYPT_COST` не задан, используется значение 12. Если `APP_ADDR` не задан, сервер запускается на порту 8080. Отсутствие `DB_DSN` считается ошибкой конфигурации, поскольку без подключения к базе данных сервис не сможет выполнять регистрацию и вход.

7. Создание модели пользователя

Создадим файл `internal/core/user.go`:

```
package core

import "time"

type User struct {
        ID           int64     `gorm:"primaryKey" json:"id"`
        Email        string    `gorm:"uniqueIndex;size:255;not null"
json:"email"`
        PasswordHash string    `gorm:"size:255;not null" json:"-"`
        CreatedAt    time.Time `json:"createdAt"`
        UpdatedAt    time.Time `json:"updatedAt"`
}
```

Модель описывает запись пользователя в базе данных. Поле `ID` является первичным ключом. Для `Email` задаётся уникальный индекс, который не позволяет зарегистрировать несколько пользователей с одним адресом. Поле `PasswordHash` предназначено для хранения bcrypt-хэша.

Тег `json:"-"` запрещает включать `PasswordHash` в JSON-ответы. Это важно, поскольку хэш пароля не должен передаваться клиенту даже после успешной регистрации или входа.

8. Подключение к PostgreSQL

Создадим файл `internal/repo/postgres.go`:

```
package repo
```

```
import (
        "context"
        "fmt"

        "gorm.io/driver/postgres"
        "gorm.io/gorm"
)

func Open(ctx context.Context, dsn string) (*gorm.DB, error) {
        db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{
                TranslateError: true,
        })
        if err != nil {
                return nil, fmt.Errorf("open PostgreSQL connection: %w", err)
        }

        sqlDB, err := db.DB()
        if err != nil {
                return nil, fmt.Errorf("get underlying SQL connection: %w",
err)
        }
        sqlDB.SetMaxOpenConns(10)
        sqlDB.SetMaxIdleConns(5)

        if err := sqlDB.PingContext(ctx); err != nil {
                _ = sqlDB.Close()
                return nil, fmt.Errorf("ping PostgreSQL: %w", err)
        }

        return db, nil
}
```

Функция `Open` устанавливает соединение с PostgreSQL, получает объект стандартного SQL-пула и проверяет доступность базы данных с помощью `PingContext`. Параметр `TranslateError: true` позволяет GORM преобразовывать ошибки нарушения уникальности в `gorm.ErrDuplicatedKey`, чтобы приложение могло корректно обработать повторную регистрацию email.

9. Создание репозитория пользователей и миграции

Создадим файл `internal/repo/user_repo.go`:

```
package repo

import (
```

```
        "context"
        "errors"

        "example.com/pz9-auth/internal/core"
        "gorm.io/gorm"
)

var (
        ErrUserNotFound = errors.New("user not found")
        ErrEmailTaken   = errors.New("email already in use")
)

type UserRepo struct {
        db *gorm.DB
}

func NewUserRepo(db *gorm.DB) *UserRepo {
        return &UserRepo{db: db}
}

func (r *UserRepo) AutoMigrate() error {
        return r.db.AutoMigrate(&core.User{})
}

func (r *UserRepo) Create(ctx context.Context, user *core.User) error {
        if err := r.db.WithContext(ctx).Create(user).Error; err != nil {
                if errors.Is(err, gorm.ErrDuplicatedKey) {
                        return ErrEmailTaken
                }
                return err
        }
        return nil
}

func (r *UserRepo) ByEmail(ctx context.Context, email string) (core.User,
error) {
        var user core.User
        err := r.db.WithContext(ctx).Where("email = ?",
email).First(&user).Error
        if errors.Is(err, gorm.ErrRecordNotFound) {
                return core.User{}, ErrUserNotFound
        }
        return user, err
}
```

Метод `AutoMigrate` создаёт таблицу на основе структуры `User`, если она ещё не существует. Метод `Create` сохраняет пользователя и преобразует ошибку уникальности email в `ErrEmailTaken`. Метод `ByEmail` ищет пользователя по адресу электронной почты и возвращает `ErrUserNotFound`, если запись отсутствует.

В запросе поиска используется параметризованное условие `Where("email = ?", email)`. Значение email передаётся отдельно от SQL-выражения, что помогает избежать SQL-инъекций.

10. Реализация регистрации и входа

Создадим файл `internal/http/handlers/auth.go`. В нём будут находиться обработчики двух маршрутов, структуры входных и выходных данных, проверка JSON и функции формирования ответов.

Полное содержимое файла:

```
package handlers

import (
        "context"
        "encoding/json"
        "errors"
        "io"
        "net/http"
        "net/mail"
        "strings"
        "unicode/utf8"

        "example.com/pz9-auth/internal/core"
        "example.com/pz9-auth/internal/repo"
        "golang.org/x/crypto/bcrypt"
)

const maxRequestBody = 1 << 20
const maxBcryptPasswordBytes = 72

type UserStore interface {
        Create(ctx context.Context, user *core.User) error
        ByEmail(ctx context.Context, email string) (core.User, error)
}

type AuthHandler struct {
        Users      UserStore
        BcryptCost int
```

```
}

type credentialsRequest struct {
        Email    string `json:"email"`
        Password string `json:"password"`
}

type responseUser struct {
        ID    int64  `json:"id"`
        Email string `json:"email"`
}

type authResponse struct {
        Status string        `json:"status"`
        User   *responseUser `json:"user,omitempty"`
}

type errorResponse struct {
        Error string `json:"error"`
}

func (h *AuthHandler) Register(w http.ResponseWriter, r *http.Request) {
        var input credentialsRequest
        if err := decodeJSON(w, r, &input); err != nil {
                writeErr(w, http.StatusBadRequest, "invalid_json")
                return
        }

        input.Email = normalizeEmail(input.Email)
        if !validEmail(input.Email) ||
                utf8.RuneCountInString(input.Password) < 8 ||
                len(input.Password) > maxBcryptPasswordBytes {
                writeErr(w, http.StatusBadRequest,
"invalid_email_or_password")
                return
        }

        hash, err := bcrypt.GenerateFromPassword(
                []byte(input.Password),
                h.BcryptCost,
        )
        if err != nil {
                writeErr(w, http.StatusInternalServerError, "hash_failed")
                return
        }
```

```
        user := core.User{
                Email:        input.Email,
                PasswordHash: string(hash),
        }
        if err := h.Users.Create(r.Context(), &user); err != nil {
                if errors.Is(err, repo.ErrEmailTaken) {
                        writeErr(w, http.StatusConflict, "email_taken")
                        return
                }
                writeErr(w, http.StatusInternalServerError, "db_error")
                return
        }

        writeJSON(w, http.StatusCreated, authResponse{
                Status: "ok",
                User:   &responseUser{ID: user.ID, Email: user.Email},
        })
}

func (h *AuthHandler) Login(w http.ResponseWriter, r *http.Request) {
        var input credentialsRequest
        if err := decodeJSON(w, r, &input); err != nil {
                writeErr(w, http.StatusBadRequest, "invalid_json")
                return
        }

        input.Email = normalizeEmail(input.Email)
        if !validEmail(input.Email) ||
                input.Password == "" ||
                len(input.Password) > maxBcryptPasswordBytes {
                writeErr(w, http.StatusBadRequest,
"email_and_password_required")
                return
        }

        user, err := h.Users.ByEmail(r.Context(), input.Email)
        if err != nil {
                if errors.Is(err, repo.ErrUserNotFound) {
                        writeErr(w, http.StatusUnauthorized,
"invalid_credentials")
                        return
                }
                writeErr(w, http.StatusInternalServerError, "db_error")
                return
        }
```

```
        if err := bcrypt.CompareHashAndPassword(
                []byte(user.PasswordHash),
                []byte(input.Password),
        ); err != nil {
                writeErr(w, http.StatusUnauthorized, "invalid_credentials")
                return
        }

        writeJSON(w, http.StatusOK, authResponse{
                Status: "ok",
                User:   &responseUser{ID: user.ID, Email: user.Email},
        })
}

func normalizeEmail(email string) string {
        return strings.ToLower(strings.TrimSpace(email))
}

func validEmail(email string) bool {
        if email == "" || len(email) > 255 {
                return false
        }
        parsed, err := mail.ParseAddress(email)
        return err == nil && parsed.Address == email
}

func decodeJSON(w http.ResponseWriter, r *http.Request, dst any) error {
        decoder := json.NewDecoder(http.MaxBytesReader(w, r.Body,
maxRequestBody))
        decoder.DisallowUnknownFields()

        if err := decoder.Decode(dst); err != nil {
                return err
        }
        if err := decoder.Decode(&struct{}{}); !errors.Is(err, io.EOF) {
                return errors.New("request body must contain exactly one JSON
object")
        }
        return nil
}

func writeJSON(w http.ResponseWriter, status int, value any) {
        w.Header().Set("Content-Type", "application/json; charset=utf-8")
        w.WriteHeader(status)
        _ = json.NewEncoder(w).Encode(value)
}
```

```
func writeErr(w http.ResponseWriter, status int, message string) {
        writeJSON(w, status, errorResponse{Error: message})
}
```

В обработчике `Register` сначала декодируется JSON и проверяется корректность email и пароля. Email очищается от пробелов по краям и приводится к нижнему регистру. Минимальная длина пароля — 8 символов. Дополнительно проверяется ограничение bcrypt: пароль не должен превышать 72 байта.

Основная операция хэширования выполняется вызовом:

```
hash, err := bcrypt.GenerateFromPassword(
    []byte(input.Password),
    h.BcryptCost,
)
```

В базу данных записывается строка `hash`, а не исходный пароль. Если адрес уже зарегистрирован, обработчик возвращает `409 Conflict`. При успешной регистрации возвращается `201 Created`.

В обработчике `Login` сначала выполняется поиск пользователя по email. Затем вызывается:

```
err := bcrypt.CompareHashAndPassword(
    []byte(user.PasswordHash),
    []byte(input.Password),
)
```

Если пользователь не найден или пароль не совпадает, клиент получает одинаковый ответ `401 Unauthorized` с ошибкой `invalid_credentials`. Это не позволяет определить по ответу сервера, зарегистрирован ли указанный email. При успешной проверке возвращается `200 OK`.

Функция `decodeJSON` ограничивает размер тела запроса, запрещает неизвестные поля и проверяет, что в теле находится ровно один JSON-объект.

11. Создание точки входа и настройка маршрутов

Создадим файл `cmd/api/main.go`:

```
package main
```

```
import (
        "context"
        "errors"
        "log"
        "net/http"
        "os"
        "os/signal"
        "syscall"
        "time"

        "github.com/go-chi/chi/v5"
        "github.com/go-chi/chi/v5/middleware"

        "example.com/pz9-auth/internal/http/handlers"
        "example.com/pz9-auth/internal/platform/config"
        "example.com/pz9-auth/internal/repo"
)

func main() {
        cfg, err := config.Load()
        if err != nil {
                log.Fatal("configuration:", err)
        }

        startupCtx, cancelStartup := context.WithTimeout(
                context.Background(),
                10*time.Second,
        )
        db, err := repo.Open(startupCtx, cfg.DBDSN)
        cancelStartup()
        if err != nil {
                log.Fatal("database:", err)
        }

        sqlDB, err := db.DB()
        if err != nil {
                log.Fatal("database pool:", err)
        }
        defer func() {
                if err := sqlDB.Close(); err != nil {
                        log.Printf("close database connection: %v", err)
                }
        }()

        users := repo.NewUserRepo(db)
        if err := users.AutoMigrate(); err != nil {
```

```
		log.Fatal("database migration:", err)
	}

	auth := &handlers.AuthHandler{
		Users:      users,
		BcryptCost: cfg.BcryptCost,
	}

	router := chi.NewRouter()
	router.Use(middleware.RequestID)
	router.Use(middleware.RealIP)
	router.Use(middleware.Logger)
	router.Use(middleware.Recoverer)

	router.Get("/health", func(w http.ResponseWriter, _ *http.Request) {
		w.WriteHeader(http.StatusOK)
		_, _ = w.Write([]byte("ok"))
	})
	router.Post("/auth/register", auth.Register)
	router.Post("/auth/login", auth.Login)

	server := &http.Server{
		Addr:              cfg.Addr,
		Handler:           router,
		ReadHeaderTimeout: 5 * time.Second,
	}

	stopCtx, stop := signal.NotifyContext(
		context.Background(),
		os.Interrupt,
		syscall.SIGTERM,
	)
	defer stop()

	serverErrors := make(chan error, 1)
	go func() {
		log.Printf("HTTP server is listening on %s", cfg.Addr)
		serverErrors <- server.ListenAndServe()
	}()

	select {
	case err := <-serverErrors:
		if !errors.Is(err, http.ErrServerClosed) {
			log.Fatal("HTTP server:", err)
		}
	case <-stopCtx.Done():
```

```
                shutdownCtx, cancel := context.WithTimeout(
                        context.Background(),
                        10*time.Second,
                )
                defer cancel()
                if err := server.Shutdown(shutdownCtx); err != nil {
                        log.Printf("graceful shutdown failed: %v", err)
                }
        }
}
```

В `main.go` последовательно выполняются загрузка конфигурации, подключение к PostgreSQL, автоматическая миграция таблицы пользователей, создание обработчика авторизации и регистрация маршрутов. После этого запускается HTTP-сервер.

Дополнительно добавлен маршрут `GET /health`, который возвращает `ok`. Он нужен для простой проверки доступности сервиса и не заменяет два основных маршрута задания.

12. Настройка PostgreSQL

Для локального запуска можно использовать Docker Compose. В корне проекта создадим файл `docker-compose.yml`:

```
services:
  db:
    image: postgres:17-alpine
    container_name: pz9-auth-postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: pz9_auth
      POSTGRES_USER: pz9_user
      POSTGRES_PASSWORD: pz9_password
    ports:
      - "5433:5432" # избегаем конфликтов с локальным PostgreSQL
                                    # для перезапуска docker используйте
команду ниже если конфликт появился
    volumes:
      - pz9_auth_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U pz9_user -d pz9_auth"]
      interval: 5s
      timeout: 3s
      retries: 10
```

```
volumes:
  pz9_auth_data:
```

```
docker compose up -d --force-recreate db
```

Запустим PostgreSQL:

```
docker compose up -d db
```

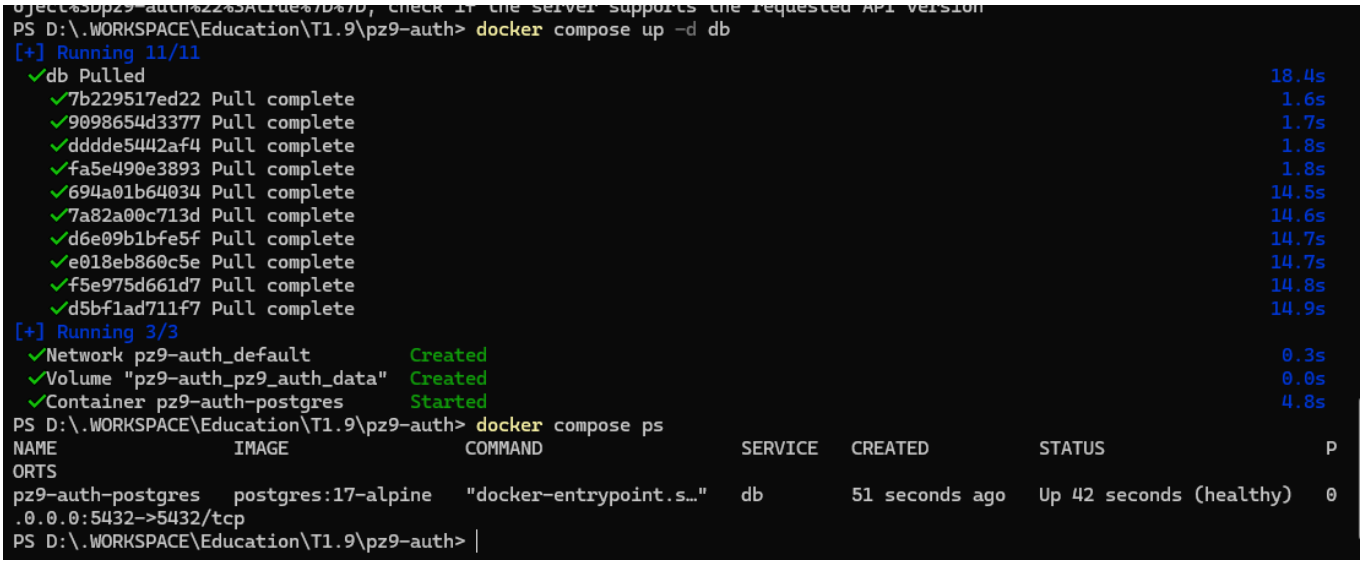
Проверим состояние контейнера:

```
docker compose ps
```

При использовании локально установленного PostgreSQL вместо Docker необходимо создать базу данных `pz9_auth` и пользователя `pz9_user` с паролем `pz9_password`. Это можно сделать в pgAdmin через Query Tool:

```
CREATE USER pz9_user WITH PASSWORD 'pz9_password';
CREATE DATABASE pz9_auth OWNER pz9_user;
```

Если пользователь и база уже существуют, повторно создавать их не нужно.



Иногда Docker не ставит пароль в образе, поэтому нужно зайти в образ PostgreSQL и назначить пароль:

```
docker exec -it pz9-auth-postgres psql -U pz9_user -d pz9_auth
# Должны увидеть pz9_auth=#
```

```
ALTER USER pz9_user WITH PASSWORD 'pz9_password';
```

13. Настройка переменных окружения и запуск приложения

Перед запуском зададим параметры подключения к базе данных, сложность bcrypt и адрес сервера.

Windows PowerShell:

```
$env:DB_DSN = 'postgres://pz9_user:pz9_password@127.0.0.1:5433/pz9_auth?
sslmode=disable'
$env:BCRYPT_COST = '12'
$env:APP_ADDR = ':8080'
```

Linux/macOS:

```
export DB_DSN='postgres://pz9_user:pz9_password@localhost:5432/pz9_auth?
sslmode=disable'
export BCRYPT_COST=12
export APP_ADDR=:8080
```

Установим и проверим зависимости:

```
go mod tidy
go vet ./...
```

Затем запустим приложение:

```
go run ./cmd/api
```

В терминале должно появиться сообщение о запуске HTTP-сервера на адресе `:8080`. Если порт занят, можно изменить `APP_ADDR` на `:8081` и использовать этот порт во всех дальнейших запросах.

Проверить доступность сервиса можно в другом окне PowerShell:

```
Invoke-RestMethod -Method Get -Uri http://localhost:8080/health
```

Ожидаемый ответ:

```
ok
```

```
[+] Running 1/1
 ✓Container pz9-auth-postgres  Started
PS D:\.WORKSPACE\Education\T1.9\pz9-auth> $env:DB_DSN = 'postgres://pz9_user:pz9_password@127.0.0.1:5433/pz9_auth?sslmode=disable'
PS D:\.WORKSPACE\Education\T1.9\pz9-auth> go run ./cmd/api
2026/10/31 11:15:28 HTTP server is listening on :8080
```

запуск приложения командой `go run ./cmd/api`.

```
PS C:\Users\Falcion> Invoke-RestMethod -Method Get -Uri http://localhost:8080/health
ok
```

успешный ответ маршрута `/health`.

14. Проверка регистрации и входа

Для проверки API можно использовать PowerShell, `curl.exe` или Postman. Ниже приведены команды для Windows PowerShell. Важно использовать именно `curl.exe`, поскольку `curl` в некоторых версиях Windows PowerShell является псевдонимом другой команды.

14.1. Успешная регистрация — 201 Created

Отправим запрос на регистрацию:

```
curl.exe -i -X POST "http://localhost:8080/auth/register" `
  -H "Content-Type: application/json" `
  --data-binary '{\"email\":\"user@example.com\",\"password\":\"Secret123!\"}'
```

Ожидаемый HTTP-статус:

```
HTTP/1.1 201 Created
```

Пример тела ответа:

```
{
  "status": "ok",
  "user": {
    "id": 1,
    "email": "user@example.com"
  }
}
```

Идентификатор пользователя может отличаться. Пароль и его хэш в ответ не включаются.

```
PS C:\Users\Falcion> curl.exe -i -X POST "http://localhost:8080/auth/register" `
>>   -H "Content-Type: application/json" `
>>   --data-raw '{"email":"user@example.com","password":"Secret123!"}'
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8
Date: Sat, 31 Oct 2026 08:17:20 GMT
Content-Length: 25

{"error":"invalid_json"}
PS C:\Users\Falcion> curl.exe -i -X POST "http://localhost:8080/auth/register" `
>>   -H "Content-Type: application/json" `
>>   --data-binary '{\"email\":\"user@example.com\",\"password\":\"Secret123!\"}'
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Date: Sat, 31 Oct 2026 08:17:49 GMT
Content-Length: 59

{"status":"ok","user":{"id":1,"email":"user@example.com"}}
```

14.2. Повторная регистрация — 409 Conflict

Повторим запрос с тем же email:

```
curl.exe -i -X POST "http://localhost:8080/auth/register" `
    -H "Content-Type: application/json" `
    --data-raw
'{\"email\":\"user@example.com\",\"password\":\"AnotherPass123!\"}'
```

Поскольку такой адрес уже существует в базе, сервер должен вернуть:

```
HTTP/1.1 409 Conflict
```

Тело ответа:

```
{
  "error": "email_taken"
}
```

```
PS C:\Users\Falcion> curl.exe -i -X POST "http://localhost:8080/auth/register" `
>>     -H "Content-Type: application/json" `
>>     --data-raw '{\"email\":\"user@example.com\",\"password\":\"AnotherPass123!\"}'
HTTP/1.1 409 Conflict
Content-Type: application/json; charset=utf-8
Date: Sat, 31 Oct 2026 08:25:09 GMT
Content-Length: 24

{"error":"email_taken"}
PS C:\Users\Falcion>
```

14.3. Вход с правильными данными — 200 OK

Теперь выполним вход с ранее зарегистрированными учётными данными:

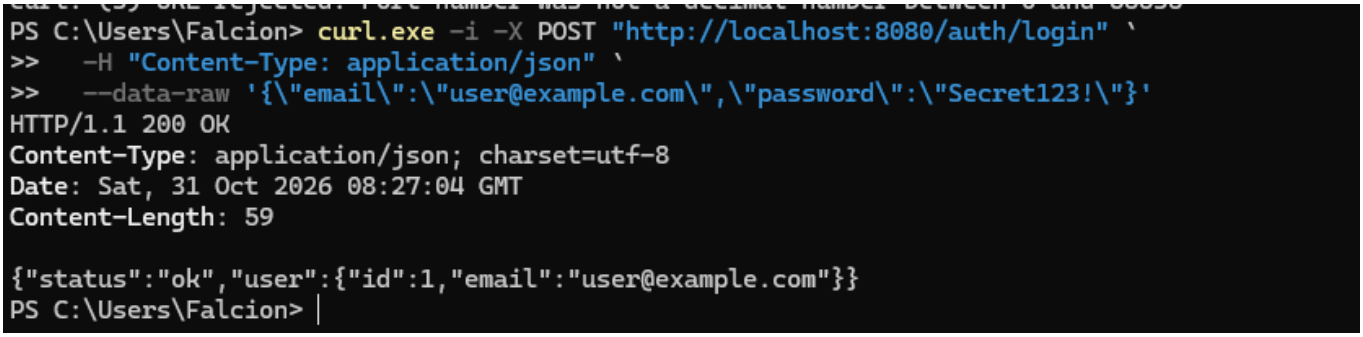
```
curl.exe -i -X POST "http://localhost:8080/auth/login" `
  -H "Content-Type: application/json" `
  --data-raw '{\"email\":\"user@example.com\",\"password\":\"Secret123!\"}'
```

Ожидаемый HTTP-статус:

```
HTTP/1.1 200 OK
```

Пример ответа:

```
{
  "status": "ok",
  "user": {
    "id": 1,
    "email": "user@example.com"
  }
}
```



14.4. Вход с неправильным паролем — 401 Unauthorized

Отправим запрос с правильным email, но неправильным паролем:

```
curl.exe -i -X POST "http://localhost:8080/auth/login" `
  -H "Content-Type: application/json" `
  --data-raw '{\"email\":\"user@example.com\",\"password\":\"WrongPass123!\"}'
```

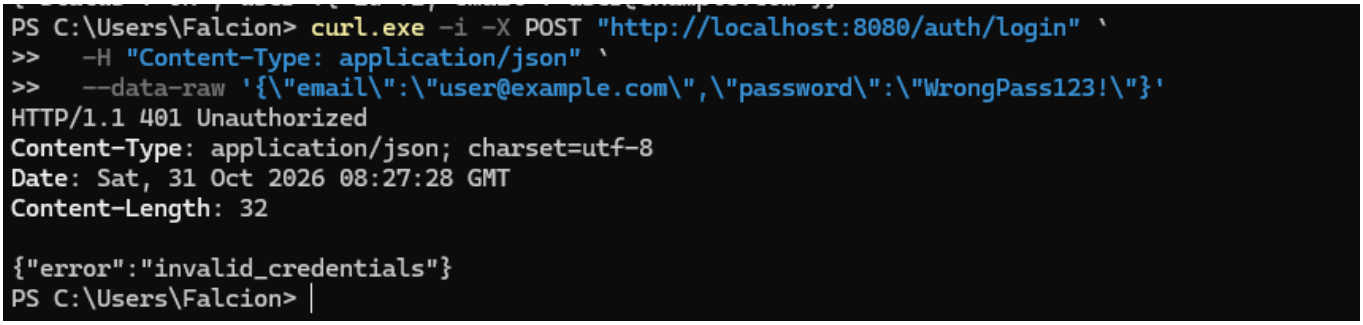
Ожидаемый HTTP-статус:

```
HTTP/1.1 401 Unauthorized
```

Тело ответа:

```
{
  "error": "invalid_credentials"
}
```

Для неизвестного email сервер возвращает такой же статус и такой же код ошибки. Это сделано намеренно, чтобы нельзя было проверять существование аккаунтов по ответам сервера.



15. Проверка данных непосредственно в PostgreSQL

После регистрации пользователя проверим, что запись действительно появилась в базе данных. Подключиться к PostgreSQL через Docker можно командой:

```
docker exec -it pz9-auth-postgres psql -U pz9_user -d pz9_auth
```

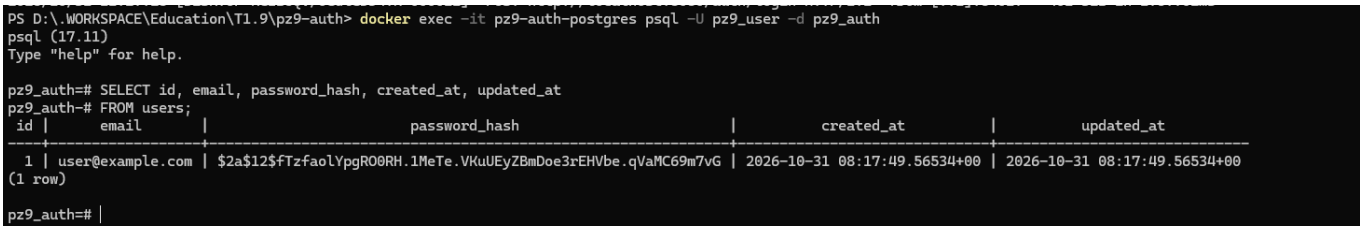
В открывшейся консоли PostgreSQL выполним:

```
SELECT id, email, password_hash, created_at, updated_at
FROM users;
```

В результате должна отображаться запись зарегистрированного пользователя. Значение `password_hash` должно быть bcrypt-хэшем, а не исходным паролем `Secret123!`. Строка bcrypt обычно начинается с `$2a$`, `$2b$` или другого допустимого идентификатора версии и содержит параметр cost.

Для выхода из `psql` используется команда:

```
\q
```



16. Запуск автоматических тестов

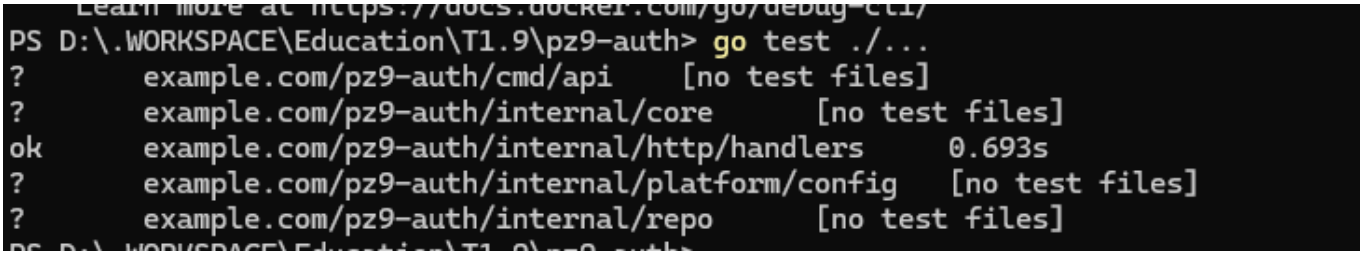
Дополнительно проверим обработчики с помощью встроенного тестового инструментария Go:

```
go test ./...
```

Тесты проверяют создание пользователя и сохранение хэша, отказ при повторном email, валидацию некорректных данных, успешный вход и одинаковую ошибку для неизвестного email и неверного пароля.

Для подробного вывода:

```
go test -v ./...
```



17. Основные HTTP-ответы сервиса

Маршрут	Сценарий	HTTP-статус
`POST /auth/register`	Пользователь успешно создан	`201 Created`
`POST /auth/register`	Email уже занят	`409 Conflict`
`POST /auth/register`	Некорректный JSON или данные	`400 Bad Request`
`POST /auth/login`	Правильные email и пароль	`200 OK`
`POST /auth/login`	Неверный пароль или неизвестный email	`401 Unauthorized`

Маршрут	Сценарий	HTTP-статус
`POST /auth/login`	Некорректный JSON или данные	`400 Bad Request`
Оба маршрута	Внутренняя ошибка базы данных	`500 Internal Server Error`

18. Ответы на контрольные вопросы

1. В чём разница между хранением пароля и хранением его хэша? Зачем нужна соль? Почему bcrypt, а не SHA-256?

При хранении пароля в открытом виде в базе данных находится исходная строка, которую можно сразу использовать для входа. При хранении хэша в базе находится результат одностороннего преобразования, а не сам пароль.

Соль — случайное значение, которое используется при хэшировании. Она нужна, чтобы одинаковые пароли не давали одинаковые хэши. bcrypt автоматически генерирует соль и включает её в итоговую строку.

SHA-256 является быстрой универсальной хэш-функцией. Из-за высокой скорости злоумышленник может проверять большое количество вариантов паролей. bcrypt специально предназначен для хранения паролей и позволяет регулировать вычислительную сложность через параметр `cost`.

2. Что произойдёт при снижении или повышении cost у bcrypt? Как подобрать значение?

При увеличении `cost` создание и проверка хэша становятся медленнее, поэтому массовый подбор паролей требует больше вычислительных ресурсов. При уменьшении `cost` проверка становится быстрее, но устойчивость к перебору снижается.

Значение следует подбирать с учётом производительности сервера и допустимого времени авторизации. В методических указаниях для учебной работы рекомендуется `cost = 12`; обычно рассматриваются значения от 10 до 14.

3. Какие статусы и ответы должны возвращать POST /auth/register и POST /auth/login в типичных сценариях?

Для регистрации при успешном создании пользователя возвращается `201 Created`. Если email уже занят, возвращается `409 Conflict`. При некорректных входных данных возвращается `400 Bad Request`.

Для входа при правильных данных возвращается `200 OK`. При неправильном пароле или неизвестном пользователе возвращается `401 Unauthorized`. В данной работе оба случая

имеют одинаковый код ошибки `invalid_credentials`.

4. Какие риски несут подробные сообщения об ошибках при логине?

Если сервер сообщает, что email существует, но пароль неверный, злоумышленник может проверять наличие аккаунтов в системе. Это упрощает сбор списка зарегистрированных пользователей и последующие атаки на них. Поэтому при неизвестном email и неправильном пароле сервер должен возвращать одинаковое сообщение об ошибке.

5. Почему в этом практическом занятии не выдаётся токен и что изменится в ПЗ №10 (JWT)?

Текущая работа посвящена регистрации, проверке пароля и безопасному хранению учётных данных. После успешного входа сервер пока возвращает обычный ответ `ok`.

В следующем практическом занятии будет добавлена JWT-аутентификация. После успешной проверки email и пароля сервер сможет сформировать токен, подписать его секретным ключом и вернуть клиенту. Клиент будет передавать токен при обращении к защищённым маршрутам, а сервер — проверять его подпись и срок действия.

19. Вывод

В ходе практической работы был разработан HTTP-сервис регистрации и входа пользователей на языке Go. Для хранения учётных записей использована база данных PostgreSQL, а взаимодействие с ней реализовано с помощью GORM.

Были созданы модель пользователя, репозиторий для работы с базой данных, конфигурация приложения и HTTP-обработчики. Реализованы маршруты `POST /auth/register` и `POST /auth/login`. При регистрации пароль преобразуется в bcrypt-хэш с параметром `cost = 12`, а в базе данных сохраняется именно хэш, а не исходный пароль.

Также реализованы проверка входных данных, обработка повторной регистрации, единый ответ при неверных учётных данных и автоматическое создание таблицы пользователей. Проверка основных сценариев выполняется HTTP-запросами и автоматическими тестами.

Таким образом, в результате работы получена основа для дальнейшего добавления JWT-аутентификации в следующем практическом занятии.